

Are Joins over LSM-trees Ready?

Take RocksDB as an Example

Weiping Yu*
Fan Wang*
weiping001@e.ntu.edu.sg
FAN008@e.ntu.edu.sg
Nanyang Technological University

Xuwei Zhang
Nanyang Technological University
zhan0612@e.ntu.edu.sg

Siqiang Luo†
Nanyang Technological University
siqiang.luo@ntu.edu.sg

ABSTRACT

LSM-tree-based data stores are widely adopted in industries for their excellent performance. As data scale increases, disk-based join operations become indispensable yet costly for the database, making the selection of suitable join methods crucial for system optimization. Current LSM-based stores generally adhere to conventional relational database practices and support only a limited number of join methods. However, the LSM-tree delivers distinct read and write efficiency compared to the relational databases, which could accordingly impact the performance of various join methods. Therefore, it is necessary to reconsider the selection of join methods in this context to fully explore the potential of various join algorithms and index designs. In this work, we present a systematic study and an exhaustive benchmark for joins over LSM-trees. We define a configuration space for join methods, encompassing various join algorithms, secondary index types, and consistency strategies. We also summarize a theoretical analysis to evaluate the overhead of each join method for an in-depth understanding. Furthermore, we implement all join methods in the configuration space on a unified platform and compare their performance through extensive experiments. Our theoretical and experimental results yield several insights and takeaways tailored to joins in LSM-based stores that aid developers in choosing proper join methods based on their working conditions.

PVLDB Reference Format:
Weiping Yu, Fan Wang, Xuwei Zhang, and Siqiang Luo. Are Joins over LSM-trees Ready?. PVLDB, 14(4): 1077 - 1090, 2024.
doi:10.14778/3717755.3717767

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/we1pingyu/lsmjoin.

1 INTRODUCTION

Log-structured merge (LSM) trees [62] based key-value stores have gained significant traction in the industry. Notable examples include RocksDB [25] at Facebook, LevelDB[28] and BigTable [13] at Google, HBase [1] and Cassandra [5] at Apache, X-Engine [30] at Alibaba, WiredTiger [2] at MongoDB, and Dynamo [23] at Amazon. These LSM-based stores are essential for applications such as social media [6, 10], stream processing [12, 16], and file systems [37, 69].

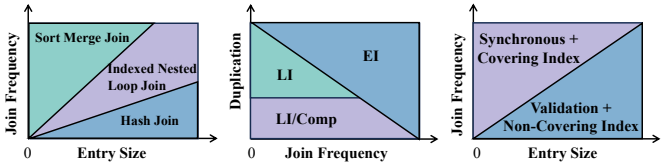


Figure 1: Our benchmark provides helpful guidelines to assist practitioner in selecting proper methods according to their working conditions for join in LSM-based key-value stores.

Joins over LSM-based stores. Many renowned LSM-based stores have implemented join operations [5, 11, 23, 25, 57, 71] as shown in Table 1. However, they typically rely on insights from relational databases to focus on a few join methods. This raises the question: Are these methods still the most appropriate choice for LSM-based stores? Our analysis suggests a likely negative answer due to the distinct and complex impact of LSM-trees. Specifically, while LSM-tree enhances updates and lookups efficiency, it requires additional costs to maintain consistency, and the overall impact of LSM-tree has not been meticulously evaluated. Consequently, there may exist substantial space for optimizing the join method selection strategy in LSM-based stores due to two problems.

Problem 1: Many join methods remain unexplored in existing LSM-based stores. As indicated in Table 1, current LSM-based databases only practice limited join methods. However, the potential of other join techniques has not been fully explored. For instance, most LSM-based stores employ a synchronous consistency strategy instead of a validation one for handling LSM-trees' out-of-place updates. This rationale is supported by the synchronous strategy's advantage in join efficiency, as the validation strategy incurs additional joining overhead. However, synchronous strategy simultaneously introduces substantial update overhead which can conversely deteriorate the performance under certain conditions. In such cases, the validation strategy may surprisingly provide better performance. Thus, it is crucial to expand the join design space to include more join methods to enhance the selection criteria.

Problem 2: Many influential factors, tied to LSM-tree properties, on join performance have not been fully examined. While existing LSM-based stores typically select join algorithms based on selectivity inspired by relational database practices [43, 70, 73, 75], LSM-tree storage introduces additional performance considerations. For instance, the performance of indexed nested

*Both authors contributed equally to this research.
†Corresponding Author

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 4 ISSN 2150-8097.
doi:10.14778/3717755.3717767

Table 1: Different join techniques supported in different renowned LSM-based key-value databases.

NoSQL Storage Systems	Secondary Index	Join Algorithm	Consistency Strategy	Covering Index
AsterixDB [4]	Embedded Index, Composite Keys	INLJ, SJ, HJ	Synchronous	✓
Cassandra [5]	Lazy Index	×	Synchronous	×
CockroachDB [71], PolarDB-X Engine [11]	Composite Index	INLJ, SJ, HJ	Synchronous	✓
HBase [1], LevelDB [28], RocksDB [25]	×	×	×	×
MyRocks [57]	Composite Index	INLJ	Synchronous	✓

loop join (INLJ) maintains robustness, compared to hash join (HJ) and sort-merge join (SJ), for large entry sizes due to the lookup optimization techniques in LSM-tree (e.g. Bloom filters). Hence, though INLJ is generally less preferred in high-selectivity cases, it still can outperform HJ and SJ when handling large entry sizes. Therefore, selecting an appropriate join method in LSM-based stores is complex and requires examining more influencing factors.

To address these issues, we systematically study joins in LSM-based stores and introduce an exhaustive benchmark that yields novel insights, contributing primarily in the following aspects:

We identify key join method characteristics and propose an inclusive configuration space covering both existing and potential new combinations. This configuration space includes four primitive join components, join algorithm (i.e. indexed nested loop join, sort-merge join, and hash join), secondary index (i.e. eager index, lazy index, and composite index), consistency strategy (i.e. synchronous strategy and validation strategy), and covering index (i.e. covering and non-covering). It allows us to describe existing join methods and discover novel ones, such as the indexed nested loop join combined with a non-covering eager index and validation strategy. Such an integrated study allows us to explore the join method overlooked by existing literature, which typically concentrates on certain aspects of the entire space [47, 65, 72].

We tailor the theoretical analysis to the joins over LSM-tree including 3 join algorithms across 12 scenarios and 6 index designs, many of which have not been previously analyzed in existing works. Different from existing join analysis, our assessment incorporates the unique characteristics of LSM-trees to evaluate the overhead of join methods. Additionally, we analyze the integrated cost of each join method in our configuration space, considering its distinct join algorithm, secondary index type, consistency strategy, and covering index. Many combinations, such as the integration of eager index designs with validation strategies, have not been explored in prior research. This thorough analysis not only leads to an in-depth understanding of the features of each join method but also guides the design of our experiments.

We implement all 29 join methods within our configuration space on a unified platform and examine them under diverse working conditions to derive guidelines for join method selection in LSM-based stores. We examine 10 factors related to workload and LSM-tree configuration to assess their impact on the performance of diverse join methods. Excitingly, this leads to several useful guidelines as Figure 1 illustrates. In the *Movie* dataset with moderate selectivity, SJ is the preferred method based solely on selectivity. However, as entry size increases to 4096 bytes, INLJ outperforms both SJ and HJ by 70%, making it the best choice. Notably, some differ from traditional relational databases and other data storage systems like those based on B+ trees. For example, while selectivity is typically the main criterion in relational databases, our results show that join frequency and entry size also play crucial roles. In the *Movie* dataset with moderate selectivity, SJ is the preferred method based solely on selectivity. However, as entry size increases to 4096 bytes, INLJ outperforms both SJ and HJ by 70%, making it the best choice. Additionally, some overlooked secondary index designs can offer competitive performance. For instance, while the eager index is usually avoided due to its high construction cost, it excels in frequent join workloads. In the *User* dataset, INLJ with a composite index performs better than the eager index when joins occur less than once per 10 million updates. However, when join frequency increases to 32, the eager index outperforms the composite index by over 30% in latency.

Table 2: Notations used in this paper. Subscripts denote associated tables (e.g., L_R: the number of levels in right data table LSM-tree, $L_{R'}$: the number of levels in right index LSM-tree).

Term	Definition	Unit
R, S	Right and left data tables	
R', S'	Index tables for right and left data tables	
L	Number of levels in an LSM-trees	levels
N	Number of entries stored in an LSM-tree	entries
e	Size of entries of LSM-tree	bytes
B	Size of data block	bytes
p	False positive rate of Bloom filters	
ϵ	Ratio between the number of matched entries and total entries in the LSM-tree	
d	Average duplication frequency of join attribute value in a sequence of updates	entries

The remainder of this paper is organized as follows: Section 2 provides preliminary knowledge about LSM-trees and joins in LSM-based stores; Section 3 introduces our configuration space; the experimental results and discussion are elucidated in Section 4; Section 5 summarizes the important insights and takeaways to enhance our understanding of this topic, as well as points out some directions to guide future research.

2 BACKGROUND

This section discusses background knowledge about joins over LSM-based databases. Frequently used notations are listed in Table 2.

2.1 Log-Structured Merge Trees

An LSM-tree organizes data using an in-memory write buffer and multiple on-disk levels with capacities increasing exponentially by a size ratio T. To store N entries of size e, an LSM-tree should incorporate $L = \log_T \frac{N \cdot e}{M}$ levels, where M is the write buffer size. LSM-tree supports mainly three types of operations as follows.

Updates. LSM-tree uses an out-of-place update strategy where key-value pairs, or *entries*, are initially stored in the write buffer. When this buffer is full, the entries are flushed to disk, progressing through the LSM-tree levels as shown in Figure 2. In the worst case, an entry reaches the largest level after $L \cdot T$ compaction processes, resulting in an update cost of $O(L \cdot T \cdot \frac{e}{B})$[19, 20].

Point Lookups. A point lookup in an LSM-tree searches for an *entry* using a given *key*. It checks through all levels, returning

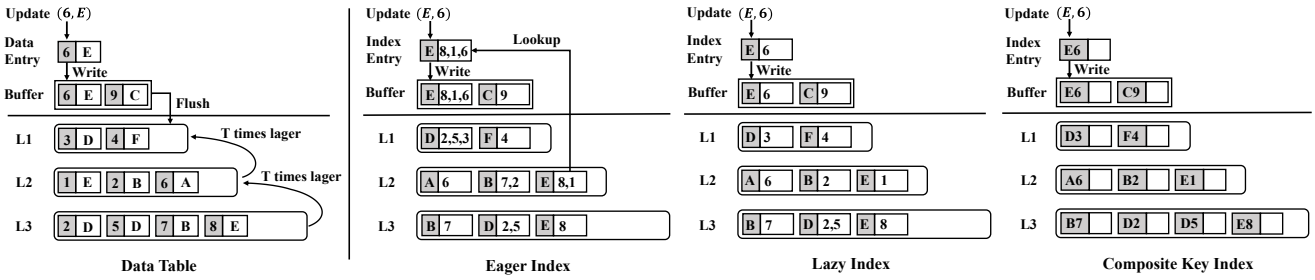


Figure 2: An illustration of LSM-trees for a data table and the corresponding index table, constructed using various secondary index types. In this figure, dark blocks represent the keys of entries, while white blocks represent the values.

the result once the matched entry is found. To speed up this process, Bloom filters [9] are employed to indicate the presence of the matched entry with false positive rate p. Thus the lookup cost is $O(L \cdot p)$ if the entry is absent, or $O(L \cdot p + \lceil \frac{e}{B} \rceil)$ if present.

Range Lookups. A range lookup retrieves entries within a key range in an LSM-tree. It incurs $O(L)$ I/O cost to seek qualified entries across all levels [19, 20], and requires an additional $O(\frac{d \cdot e}{B})$ cost for entry retrieval, where d is the number of matched entries.

2.2 Joins in LSM-tree based databases

Joins in LSM-tree based stores typically fall into two scenarios based on the alignment of the join attribute with the primary keys of the involved data tables. We examine an elementary case involving two *data tables*, where the left and right tables are denoted as R and S. The data tables are stored in separate LSM-trees with their primary keys as the keys in LSM-tree entries. We represent these tables as $R(p_r, n_r)$ and $S(p_s, n_s)$, where p is the primary key and n represents non-primary attributes. Hence, there are two possible scenarios for the data tables in join operation. In one scenario, the join attribute is the primary key of the data table, thus entries with specific join attributes can be accessed directly via the LSM key (referred to as primary index) without additional overhead. The other scenario is more complex where the join attribute is a non-primary attribute. In this case, entries are difficult to identify by the join attribute since non-primary attributes are stored in the values of the LSM-tree entries. Therefore, secondary indexes can be introduced to identify entries by join attributes and then benefiting join algorithms involving lookups (e.g., indexed nested loop joins), and the corresponding *index tables* are denoted as R' and S', respectively. Meanwhile, this introduces a tradeoff between potential join performance gains and additional index construction overhead. Our benchmark includes both scenarios and various secondary index types to provide a comprehensive analysis.

3 CONSOLIDATED JOIN CONFIGURATION SPACE FOR LSM-TREE

Algorithm 1: Update with Synchronous

Input: Data table R, index table R', update data (p_i, n_i)
Lookup p_i in R for the corresponding secondary key n_x
if n_x == $NULL$ **then**
  Insert (p_i, n_i) to R'
else
  Lookup n_x in R' for the corresponding primary key p_x
  Delete (n_x, p_x) in R'
  Insert (n_i, p_i) to R'

Our investigation reveals that existing renowned LSM-based stores support only a limited number of join methods, leaving many potentially high-performing methods unexplored. This gap indicates a lack of systematic study of join methods in LSM-based stores, for which a comprehensive configuration space encompassing a wide range of instances is necessary. To fill this gap we are the first to propose an inclusive configuration space tailored for joins over LSM-trees comprising various join algorithms, secondary indexes, consistency strategies, and covering indexes. This configuration encompasses all the existing join methods and many combinations that have never been discussed before. Additionally, we provide a theoretical analysis tailored to LSM-trees for each join method within the configuration space, which encourages a more comprehensive and thorough understanding of joins over LSM-trees.

This section provides a general introduction to and theoretical analysis of various index designs (i.e., index types, consistency strategies, and covering indexes) and join algorithms.

3.1 Secondary index types analysis

In LSM-based key-value stores, secondary indexes present a novel trade-off among join performance, update overhead, and space consumption that requires detailed analysis. We consider three secondary index types [47, 65]: *Eager Index* (Eager), *Lazy Index* (Lazy), and *Composite Index* (Comp). Our analysis in Table 3 quantitatively evaluates their distinct performance to guide the index selection for specific conditions.

Eager Index. Eager Index uses join attributes as keys and stores associated primary keys and attributes in a posting list as the value of the index entry. This structure links multiple entries that share the same join attribute value. Its update process requires an extra point lookup to retrieve the existing index entry with the same join attribute, and then updates this entry before inserting it into the index LSM-tree. It incurs $O(L \cdot p + \lceil \frac{e}{B} \rceil)$ I/O cost for retrieval and $O(L \cdot T \cdot \frac{e}{B})$ for updates, as exemplified in Figure 2. The cost of data extraction via point lookups is $O(L \cdot p)$ for empty lookups and $O(L \cdot p + \lceil \frac{e}{B} \rceil)$ for non-empty ones. The low false positive rate of Bloom filters suggests a lower cost for point lookups, potentially favoring eager indexes and indexed nested loop joins for certain workloads, as discussed in the evaluation section.

Lazy Index. Lazy Index also uses posting lists to store associated primary keys within an index entry but defers their merging. Unlike Eager Index, it merely appends a new entry to the write buffer, with index entries gradually merging during compaction at a cost of $O(L \cdot T \cdot \frac{e}{B})$. As a result, entries with the same key may exist concurrently

Table 3: The theoretical analysis of various secondary indexes and consistency strategies. In this table, the point lookup cost refers to the I/O cost of finding index entries associated with a certain join attribute value. Moreover, it is worth mentioning that we only present the cost for updating the index table given the update cost of the data table is identical for each method.

Index	Consistency strategy	Index Type	Empty Point Lookup (Z_0)	Non-empty Point Lookup (Z_1)	Update (U)
S-Eager	Synchronous	Eager Index	$O(L' \cdot p)$	$O(L' \cdot p + \lceil \frac{e'}{B} \rceil)$	$O(L \cdot p + \lceil \frac{e}{B} \rceil) + O(L' \cdot p + \lceil \frac{e'}{B} \rceil) + O(L' \cdot T \cdot \frac{e'}{B})$
S-Lazy		Lazy Index	$O(L' \cdot p)$	$O(L' \cdot \lceil \frac{e'}{B} \rceil)$	$O(L \cdot p + \lceil \frac{e}{B} \rceil) + O(L' \cdot T \cdot \frac{e'}{B})$
S-Comp		Composite Keys	$O(L' \cdot p)$	$O(L' + d \cdot \frac{e'}{B})$	$O(L \cdot p + \lceil \frac{e}{B} \rceil) + O(L' \cdot T \cdot \frac{e'}{B})$
V-Eager	Validation	Eager Index	$O(L' \cdot p)$	$O(d \cdot (L \cdot p + \lceil \frac{e}{B} \rceil)) + O(L' \cdot p + \lceil \frac{e'}{B} \rceil)$	$O(L' \cdot p + \lceil \frac{e}{B} \rceil)) + O(L' \cdot T \cdot \frac{e'}{B})$
V-Lazy		Lazy Index	$O(L' \cdot p)$	$O(d \cdot (L \cdot p + \lceil \frac{e}{B} \rceil)) + O(L' \cdot \lceil \frac{e'}{B} \rceil)$	$O(L' \cdot T \cdot \frac{e'}{B})$
V-Comp		Composite Keys	$O(L' \cdot p)$	$O(d \cdot (L \cdot p + \lceil \frac{e}{B} \rceil)) + O(L' + d \cdot \frac{e'}{B})$	$O(L' \cdot T \cdot \frac{e'}{B})$

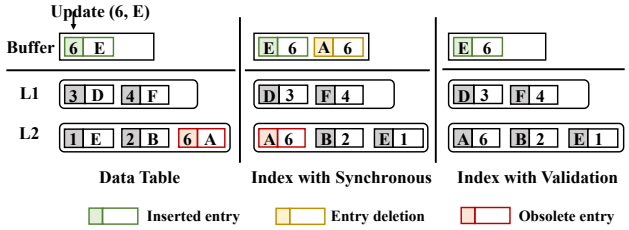


Figure 3: Synchronous incurs extra update cost, Validation cannot remove invalid entries, adding query overhead.

at different LSM-tree levels, requiring a modified lookup process that examines all levels to aggregate all relevant entries. This results in a lookup cost of $O(L \cdot \lceil \frac{e}{B} \rceil)$. For instance, as Figure 2 shows, retrieving entries with join attribute E requires extracting index entries from multiple levels to compile the complete posting list $\{6, 1, 8\}$. Lazy Index provides better update efficiency but incurs higher lookup overhead, suiting workloads requiring rapid index updates but tolerating infrequent point lookups.

Composite Index. The secondary index with a composite key stores tuples with the same join attribute value in separate entries, using a concatenated index key of the primary key and join attribute. This setup enhances lookups with prefix filters, where the join attribute serves as the prefix. For example, as illustrated in Figure 2, n update involving the primary key $\{6\}$ and join attribute $\{E\}$ generates an index key $\{E6\}$, incurring an update cost of $O(L \cdot T \cdot \frac{e}{B})$. Point lookups require examining all LSM-tree levels due to the potential distribution of entries, with a cost of $O(L + d \cdot \frac{e}{B})$, where d is the number of matched entries. Although Composite Index generally offers robust update and lookup performance, its practicality may suffer due to increased space requirements and more compaction cost, particularly with more duplicated join attributes.

Runing example: A data table with 10 million 64 bytes tuples introduces a three-level LSM-tree if the write buffer is 16 MB and size ratio is 10. When the Bloom filter is set to 10 bits per key, according to Table 3, the searching cost $(L' \cdot p)$ of Eager is neglectable compared to the other two types, indicating faster lookup performance. Meanwhile, the update cost of Eager is much higher since the writing cost $(L \cdot T \cdot \frac{e}{B})$ is less than 1 while the additional cost $(L \cdot p + \lceil \frac{e}{B} \rceil)$ is greater than 1. Hence Eager benefits query-intensive workloads like indexed nested loop join with frequent join.

3.2 Consistency strategy analysis

Due to the out-of-place update mechanism, obsolete data entries in LSM-based systems may not be promptly removed, requiring consistent maintenance between data and index tables to ensure accurate join results that are achieved via typical two methods.

Algorithm 2: Query with Validation Strategy

```
Input: Data table R, index table R', queried secondary key n_i
Lookup n_i in R' for the corresponding primary key p_x
if p_x is NULL then
    return NULL
else
    Lookup p_x in R for the corresponding secondary key n_x
    if n_x == n_i then
        return (n_i, p_i)
    else
        return NULL
```

Synchronous. Synchronous immediately checks validity upon updates to maintain consistency. This strategy improves lookup performance but increases update costs due to additional synchronization. The update process involves verifying data entry existence and updating the index. Overall update costs for different secondary indexes are detailed in Table 3.

Validation. Validation in LSM-style secondary indexes ensures join result correctness by checking the data table during joins. Hence, it simply need to insert new entry to update the index LSM-tree without removing obsolete data. Accordingly, the joining cost of Validation includes both querying the index entry and verifying all associated primary keys, expressed as $O(d \cdot (L \cdot p + \lceil \frac{e}{B} \rceil)) + O(L' + d \cdot \frac{e'}{B})$ in Table 3, where d is the number of primary key duplicates per index entry. Validation therefore incurs lower costs during index building but higher costs during joining.

Covering Index. A covering index includes the primary key and all tuple attributes, except the join attribute, within the index entry's value. Conversely, a non-covering index only stores the secondary and primary key, resulting in smaller entry sizes which delivers improved update and degraded lookup performance. The choice between covering and non-covering index is closely tied to the consistency strategy. Covering indexes complement Synchronous methods by enhancing lookup performance without querying data LSM-trees. Validation pairs well with non-covering indexes, as both require data entry retrieval.

Table 4: Theoretical analysis of join cost, and S and R represent the inner and outer table for INLJ, respectively.

Method	Join algorithm	Data table S	Data table R	Space complexity	Join cost
INLJ-P	Indexed Nested Loop Join	Primary index	Regular column	$D(R)+D(S)$	$(N_R \cdot \frac{e_R}{B}) + N_R((1-\epsilon_R)Z_0(S) + \epsilon_R \cdot Z_1(S))$
INLJ-PS		Primary index	Secondary index	$D(R)+D(S)+D(R')$	$(N_{R'} \cdot \frac{e_{R'}}{B}) + N_R((1-\epsilon_R)Z_0(S) + \epsilon_R \cdot Z_1(S))$
INLJ-N		Regular column	Regular column	$D(R)+D(S)$	$(N_R \cdot \frac{e_R}{B}) + N_R(N_S \cdot \frac{e_S}{B})$
INLJ-NS		Secondary index	Regular column	$D(R)+D(S)+D(S)$	$(N_R \cdot \frac{e_R}{B}) + N_R((1-\epsilon_R)Z_0(S') + \epsilon_R \cdot Z_1(S'))$
INLJ-SS		Secondary index	Secondary index	$D(R)+D(S)+D(R')+D(S')$	$(N_{R'} \cdot \frac{e_{R'}}{B}) + N_R((1-\epsilon_R)Z_0(S') + \epsilon_R \cdot Z_1(S'))$
SJ-P	Sort-merge Join	Primary index	Regular column	$2D(R)+2D(S)$	$(N_S \cdot \frac{e_S}{B}) + 5(N_R \cdot \frac{e_R}{B})$
SJ-PS		Primary index	Secondary index	$D(R)+2D(S)+D(R')$	$(N_S \cdot \frac{e_S}{B}) + (N_{R'} \cdot \frac{e_{R'}}{B})$
SJ-N		Regular column	Regular column	$2D(R)+2D(S)$	$5(N_S \cdot \frac{e_S}{B}) + 5(N_R \cdot \frac{e_R}{B})$
SJ-NS		Secondary index	Regular column	$2D(R)+D(S)+D(S')$	$(N_{S'} \cdot \frac{e_{S'}}{B}) + 5(N_R \cdot \frac{e_R}{B})$
SJ-SS		Secondary index	Secondary index	$D(R)+D(S)+D(R')+D(S')$	$(N_{S'} \cdot \frac{e_{S'}}{B}) + (N_{R'} \cdot \frac{e_{R'}}{B})$
HJ-P	Hash Join	Primary index	Regular column	$2D(R)+2D(S)$	$3(N_S \cdot \frac{e_S}{B}) + 3(N_R \cdot \frac{e_R}{B})$
HJ-N		Regular column	Regular column	$2D(R)+2D(S)$	$3(N_S \cdot \frac{e_S}{B}) + 3(N_R \cdot \frac{e_R}{B})$

3.3 Join algorithm analysis

In this section, we integrate the previously discussed index designs into three join algorithms: *indexed nested loop join* (INLJ), *sort-merge join* (SJ), and *hash-based join* (HJ), in LSM-based key-value stores. A join method is the combination of specific join algorithm, secondary index type, and consistency strategy.

Indexed Nested Loop Join. INLJ is a fundamental join that scans the left (outer) data table R and searches the right (inner) data table S for matching join attributes. This method's overhead includes scanning and searching costs. Scanning each entry in R's LSM-tree incurs a constant cost of $N_R \cdot \frac{e_R}{B}$. Without an index, searching requires scanning the entire right LSM-tree for each R entry, costing $N_R \cdot (N_S \cdot \frac{e_S}{B})$, which is unacceptable. However, if S is indexed, either by its primary key or a separate index LSM-tree, the cost is drastically reduced by the point lookup operation. As detailed in Section 3.1, point lookup costs depend on whether the search finds a match. Assuming a match ratio ϵ_R in R, the search cost is $(N_R \cdot \frac{e_R}{B}) + N_R((1-\epsilon_R)Z_0 + \epsilon_R \cdot Z_1)$, where Z_0 and Z_1 represent the costs of empty and non-empty lookups shown in Table 3. The overall join costs for various scenarios are further detailed in Table 4.

Sort-Merge Join. SJ identifies matching tuples by sorting join attributes from the left and right data tables. To this end, entries are scanned from the LSM-trees, sorted into multiple runs, and written back to disk. Note that single-run sorting is not feasible due to memory limits. Subsequently, these runs are processed by a k-way merge [8] and then saved to disk which is finally read out to execute the join. Hence the join cost sums up to $5(N \cdot \frac{e}{B})$.With available indexes, sorting and merging steps can be skipped since entries can be retrieved sequentially by join attributes. As shown in Table 4, SJ performs best when both tables are indexed, which our experimental results confirm.

Hash Join. HJ groups tuples with matching attributes from the left and right tables into the same buckets using join attributes. For left table R, it scans all entries, assigns them to buckets based on hash values, and stores them on disk. During the join, these buckets are retrieved, resulting in a total cost of $3(N_R \cdot \frac{e_R}{B})$. Unlike SJ, HJ's efficiency is not influenced by the presence of indexes. As shown in Table 4, HJ generally has lower join overhead than SJ, but SJ performs better when indexes are used. Further analysis of both algorithms' performance is provided in later sections.

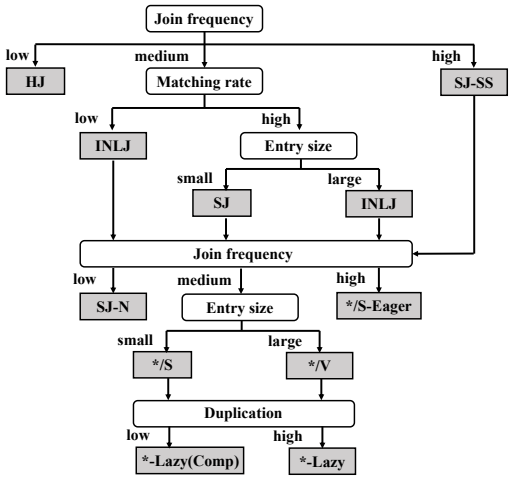


Figure 4: A sketch of a decision graph according to the theoretical analysis results in Table 3 and Table 4.

Runing example: Consider join of two data tables (*-P), both consisting of 10 million 64 bytes tuples, has matching rate of 1%. The data block size is 4KB, and Bloom filter uses 10 bits per key. According to Table 4, SJ and HJ exhibit comparable performance, while INLJ outperforms them due to efficient query operations (small Z_0 and Z_1). When e_R, the entry size of table R, decreases from 64 to 4 bytes, SJ's join cost decreases substantially until surpassing INLJ, whose performance remains stable due to the ceiling component in Z_1. Conversely, as e_R increases, HJ can outperform SJ.

4 EVALUATING JOINS OVER LSM-TREES

We experimentally evaluate the influence of various factors on the performance of diverse join methods to summarize practical guidelines for choosing the most suitable methods under specific working conditions. To this end, we identify several important variants introduced in the theoretical analysis as presented in Tables 3 and 4, which can be classified into two categories: **join algorithm related factors** and **secondary index related factors**. The following parts examine each of these factors individually.

Hardware. All experiments are conducted on a server with an Intel Xeon Gold 6326 processors, 256GB DDR4 main memory, 1TB

Table 5: Feature of Data Distribution in Benchmark Datasets.

Dataset		Duplication		Matching		Skewness	
		d_r	d_s	ϵ_r	ϵ_s	θ_r	θ_s
Real Database	*Movie*	4.1	5.6	0.72	0.99	0.89	0.85
	User	4.2	3.9	0.42	0.40	0.93	0.92
	Face	1.0	1.0	1	1	0	0
	Wiki	1.1	1.1	1	1	0.20	0.20
Synthetic Datasets	*Unif*	4.0	4.0	0.8	0.8	0	0
	Zipf	2.6	2.6	1	1	0.5	0.5

NVMe SSD, a default page size of 4 KB, and running 64-bit Ubuntu 20.04.4 LTS on an ext4 partition.

Implementation. We implement all 29 join methods within our proposed configuration space on a unified platform RocksDB [25], a prevalent LSM-based storage. While RocksDB does not inherently support joins, this required us to design custom workflows for various join algorithms and index types, many of which were developed from scratch for LSM-based stores. For instance, in the Eager and Lazy index designs, long posting lists, particularly with skewed or duplicated data, introduce significant overhead. To mitigate this and improve efficiency, we applied optimized C++ techniques for string management [58]. The LSM-tree construction for both data and index tables follows RocksDB's default settings, with a block size of 4096 bytes, a 16MB write buffer, a size ratio of 5 to introduce more levels, and no block cache. Our implementation of sort-merge join using a streamlined approach, setting the total sorted segment size to 16MB. For hash join, we integrated BKDR hash function [63] with Grace Hash Join to reduce hash collisions, using 16MB memory for partitioning and probing.

Baselines. We examine all join methods in our configuration space each specified by particular join algorithm (i.e. INLJ, SJ, and HJ), secondary index type (i.e. Eager, Lazy, and Comp), and consistency strategy (i.e. Synchronous (S) and Validation (V)). For example, INLJ-NS/S-Eager refers to the indexed nested loop join applied to a scenario where the data table R is a regular column and S is indexed using an eager index with a synchronous consistency strategy.

Datasets. We incorporate four real datasets and two synthetic datasets in our experiments as listed in Table 5. These datasets encompass diverse typical workloads with distinct data distributions specified by duplication, matching rate, and skewness. Specifically, duplication refers to the average number of replicas of primary keys (c_r, c_s) or join attributes (d_r, d_s) in the data table, matching rate (ϵ) represents the ratio of the selected entries in a join, and skewness (θ) reflects the shape of data distribution.

Following the existing works related to joins [50, 51, 66, 70, 73, 75], we examine three widely used benchmarks, Stack [52], IMDB [43], and SOSD [51] to derive our real datasets. For tailoring each benchmark to our task, we select two joinable attribute as the join attributes. *User* is derived from the Stack benchmark, which involves high duplication and skewness along with a moderate matching rate. *Movie* dataset is extracted from IMDB benchmark that has high duplication and skewness while presenting a low matching rate. *Face* and *Wiki* dataset are selected from SOSD benchmark which presents high matching rate and slightly skewed distribution with a few duplications [51], respectively.

Furthermore, we design two synthetic datasets that enable swift adjustment of features like entry size and skewness to generate diverse workloads. *Unif* dataset uses uniformly distributed integers as join attributes, while *Zipf* dataset uses widely utilized Zipfian distribution [31, 34, 46, 59, 67, 74] to generate skewed join attributes.

By default, each dataset contains 10 million entries (64 bytes per entry, with 10-byte primary keys and join attributes), resulting in a total size of 0.5GB. This setup balances capturing key LSM-tree characteristics with experimental efficiency. We also vary dataset sizes from 0.5GB to 40GB in later experiments for a broader analysis.

4.1 Current join methods can be suboptimal

Current LSM-based stores typically adhere to conventions in relational databases that determine join algorithms based on selectivity and neglect many potentially effective index designs. Our experiments indicate that several factors also significantly impact performance of different join methods, like entry size (Figure 5 (a)), join frequency (Figure 5 (b)), and matching rate (Figure 5 (c)). Furthermore, various index designs exhibit impressive performance under specific conditions, influenced by entry size (Figure 6), data distribution (Figure 7 and Figure 8), and join frequency (Figure 9). To the best of our knowledge, no existing works have thoroughly investigated the impact of these factors on the join methods selection or carefully examined the underlying principles. Our study fills this gap and provides novel insights from extensive experiments.

4.2 Entry size vs. join algorithm selection

We test different join algorithms on two real datasets, *User* and *Face* that present different performance trends, with entry sizes varying from 32 bytes to 2048 bytes. We evaluate a wide range of scenarios, including those without index, single index, and dual index, as presented in Figure 5 (a). In single index cases, we focus on configurations with a primary index to avoid the impact of index construction costs. For dual index scenarios, INLJ and HJ are excluded since, as Table 4 indicates, additional indexes do not lead to performance enhancement. Moreover, those without index typically present inferior performance, thus being placed in our technical report for space constraints. Furthermore, Composite Index and synchronous strategy are employed by default, and the impact of index types is discussed in subsequent subsections.

The results show that SJ and HJ have comparable join latencies across various cases and datasets, consistent with our theoretical analysis. The join costs of SJ-P and HJ-P are $(N_S \cdot \frac{e_S}{B}) + 5(N_R \cdot \frac{e_R}{B})$ and $3(N_S \cdot \frac{e_S}{B}) + 3(N_R \cdot \frac{e_R}{B})$, respectively, which are nearly identical when tables R and S have similar data scales and entry sizes. Both algorithms exhibit a linear increase in join cost as entry size grows, aligning with the experimental results. In contrast, INLJ shows greater resilience to larger entry sizes. For example, as entry size increases from 32 to 4096 bytes in the *Face* dataset, HJ's join latency increases sixfold, while INLJ's latency only rises by 50%. As analyzed in Section 3.3, INLJ's overhead includes scanning and searching costs, the former grows with entry size, but the latter remains constant if the entry size is smaller than the block size. This explains why INLJ, initially performing worse with small entries, gradually outperforms SJ and HJ as the entry size increases. In the *User* dataset, INLJ's latency is 3 seconds higher than SJ and HJ with 32 byte entries but surpasses both when the entry size exceeds 512 bytes.

Notably, sort-merge join with dual index (SJ-PS) exhibits impressive join efficiency due to the efficient range scan. However, it requires significant index building overhead, degrading the overall

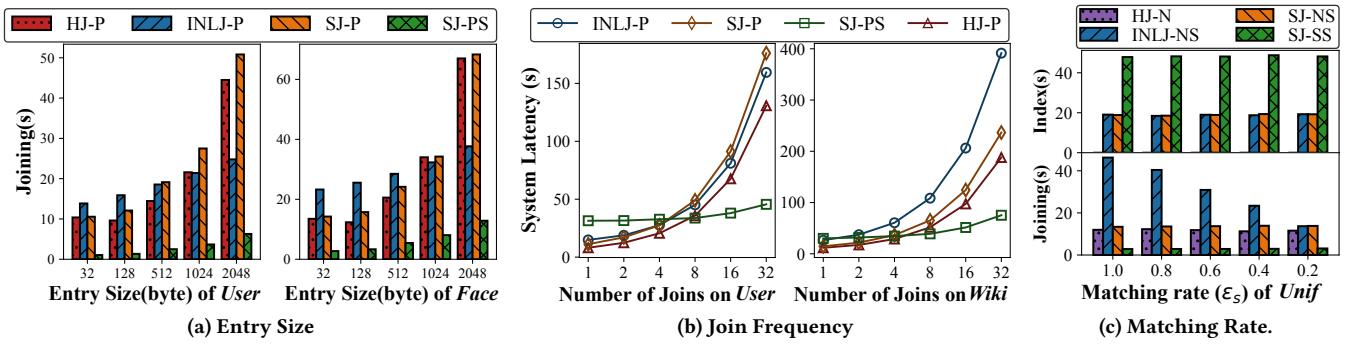


Figure 5: Join algorithm performance varies greatly with changes in entry size, join frequency, and matching rate. In (a), SJ-PS incurs additional index building latency (in seconds) as follows: 29, 43, 278, 627, 1162 (*User*); 25, 43, 239, 597, 1149 (*Face*).

performance. Meanwhile, the impact of index building cost varies with join frequency which we explore in the next subsection.

4.3 Join frequency vs. join algorithm selection

To explore the impact of join frequency on algorithm selection, we vary join frequencies from 1 to 32 and evaluate the performance of various join algorithms. The dataset selection follows the criteria with Figure 5 (a), with the inclusion of *Wiki* for diversity. Here, join frequency refers to the number of join operations in a fixed workload. For example, with a join frequency of 2, a dataset with 10 million updates is split into two subsets of 5 million updates each, and a join operation is performed after processing each subset. Thus the total number of updates remains constant across experiments, regardless of join frequency. We assessed system latency including both join and index-building latency, as shown in Figure 5 (b). The results show that SJ-PS performs poorly at low join frequencies but gradually surpasses other algorithms as frequency increases. For example, with a join frequency of 1, the system latency of SJ-PS is nearly five times that of the best-performing algorithm, hash join with primary index (HJ-P). However, with join frequency increases to 32, SJ-PS becomes the best algorithm with system latency 50% of HJ-P. As detailed in Table 4 and Figure 5 (a), SJ-PS achieves high join efficiency because its indexes allow sequential retrieval by join attributes without sorting. This comes at the cost of extra index construction overhead. As join frequency rises, the latency of SJ-P, INLJ-P, and HJ-P increases substantially, while SJ-PS grows more moderately, making it more appealing for high-frequency joins.

INLJ-P typically has higher system latency than SJ-P and HJ-P for small entry sizes, as discussed in Section 4.2. However, INLJ outperforms SJ in the *User* dataset, which challenges our expectations. Differences in matching rates, duplication, and skewness between the datasets highlight the significance of these factors in selecting join methods, which is analyzed in the following section.

4.4 Matching rate vs. join algorithm selection

As discussed in Section 4.3, matching rate can significantly impact join algorithm performance, influencing the selection of the most suitable method. Since matching rates in real datasets cannot be adjusted, we use the representative dataset *Unif* to evaluate various join algorithms, with matching rates decreasing from 1.0 to 0.2, where the join attribute is a non-primary key in both tables R and S. We break down system latency into joining and index-building latency, as shown in Figure 5 (c). It is observed that INLJ's join cost is positively correlated with the matching rate. This is because lower

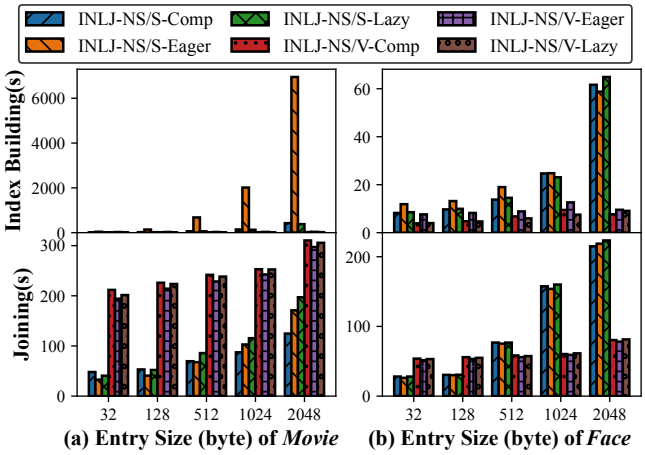


Figure 6: The Impact of entry size on index designs.

matching rates lead to fewer matched entries and more empty point lookups, which cost less than non-empty lookups, as indicated in Table 3. In contrast, HJ and SJ join latencies are unaffected by matching rate, as they must process all entries regardless of matches. This experiment also reveals the general impact of secondary indexes on the performance of different join algorithms. The update performance of secondary indexes affects the index building latency of INLJ and SJ, while the lookup performance mainly influences the joining latency of INLJ. More complete and in-depth analysis is provided in the following subsections.

Remark: Join algorithm selection is influenced by more than just selectivity. Larger entry sizes or lower matching rates favor INLJ, while higher join frequency reduces the impact of index-building overhead, making index-based algorithms like INLJ and SJ more advantageous. When index-building costs are negligible, SJ with two indexes is the preferred choice.

4.5 Entry size vs. Index design

To examine the impact of entry size on index design, we evaluated different entry sizes on real datasets *Movie* and *Face*, representing the more skewed cases and more uniform (less duplicated) scenarios, respectively. For the Eager Index with Synchronous, the index building costs increase obviously with entry size on skewed dataset *Movie*, reaching 7000 seconds for 2048-byte entries as is shown in Figure 6. This is due to the exhaustive merging of entries with the same keys. Moreover, in skewed datasets, some entries

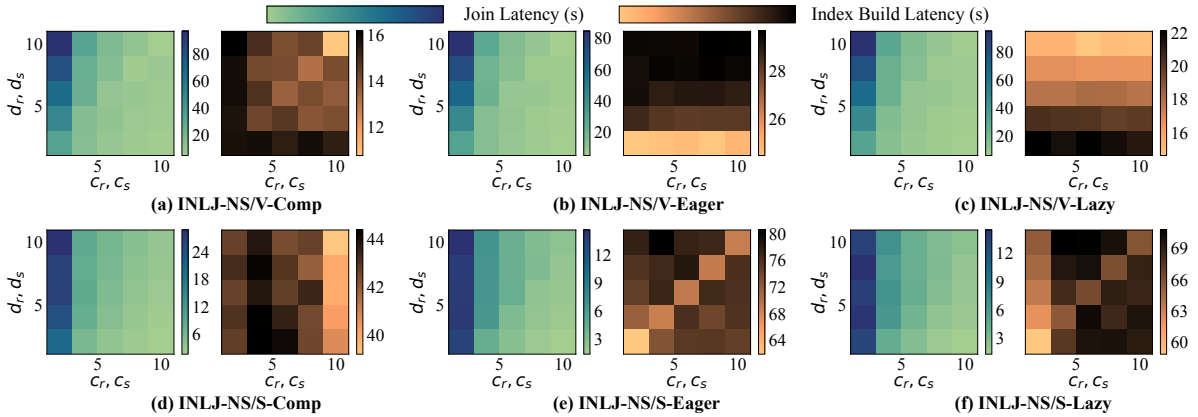


Figure 7: The performance of various index designs with diverse primary key and join attribute duplication in *Unif* dataset.

occur more frequently, leading to dramatically larger entry sizes. Synchronous has to retrieve these large entries repeatedly to remove stale values. In this process, large entry sizes can lead to a more serious burden. This is due to the frequent merging of large, repeated entries, which significantly increases the overhead of retrieving and removing stale values. In contrast, Composite and Lazy Indexes, which don't eagerly merge entries, handle entry size more efficiently, resulting in relatively stable index building costs.

In uniform datasets like *Face*, Synchronous index building costs remain stable due to fewer merges, but its join costs surpass those of Validation, since Validation avoids repeated checks in less-duplicated data. Moreover, Synchronous must maintain a large index table, with lookup overhead, as analyzed in Table 3, increases with entry size. In contrast, Validation, storing only primary keys and join attributes, has more stable index sizes, making it preferable for large entries in uniform datasets. For skewed data, the Eager Index is inefficient due to high index building time, especially with large entries. The composite key index is more favorite given it generally delivers sound performance. However, this preference is also subject to the join frequency as we introduce in Section 4.7.

4.6 Data distribution vs. Index design

We examine data distribution's impact on index design in two scenarios. Since data distribution is determined in real datasets, we include synthetic datasets to analyze the duplication in uniform datasets by varying the duplicates of primary key (c_r, c_s) and join attribute in a grid pattern on *Unif* dataset, as well as investigate the impact of join attribute skewness (θ_r, θ_s) on *Zipf* dataset.

Duplication. As Figure 7 reveals, join latency decreases with increment of primary key duplicates, since LSM-tree merges identical primary keys that effectively reduces data scale. For all Validation methods (Figures 7(a)-(c)), join costs increase with higher join attribute duplication. This aligns with our analysis in Table 3, indicating the increased lookup cost of Validation. For Eager with Validation, the cost is represented as $O(d \cdot (L \cdot p + \lceil \frac{e}{B} \rceil)) + O(L' \cdot p + \lceil \frac{e'}{B} \rceil)$. As the duplication d increases, so does the overall cost. In the Synchronous approach, only Composite Index shows a slight increase in join latency with higher join attribute duplication (Figure 7(d)). This is because Eager Index and Lazy Index can merge more identical keys into single entries, consistent with our analysis.

Regarding index building, Eager Index demonstrates increased latency with higher join attribute duplication (Figure 7(b)) due

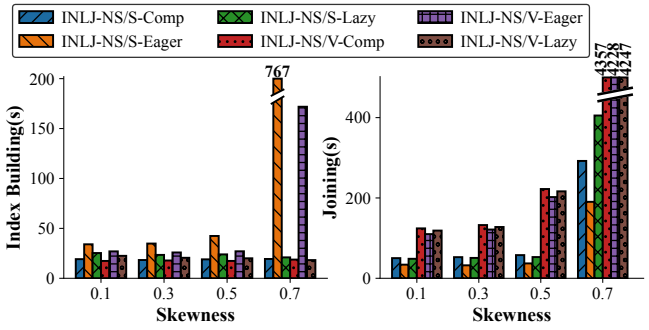


Figure 8: The performance of various index designs with varying skewness in *Zipf* dataset.

to non-empty point lookups for each duplicate. In contrast, Lazy Index shows an opposite trend (Figure 7(c)) because the compaction process in LSM-tree merges identical join attributes, which not only avoids extra lookup costs but also reduces the index table size. For Synchronous methods, the trend is less clear. Higher primary key duplication leads to more entry checks but also a smaller index table, causing index building costs to fluctuate. Consequently, for Composite Index (Figure 7(d)) and Lazy Index (Figure 7(f)), index build costs remain relatively stable.

Skewness. Figure 8 reveals striking patterns in index building and join latency as data skewness increases. Notably, Eager Index demonstrates exceptionally high index building latency, while all Validation methods exhibit dramatically high join latency. When skewness reaches 0.7, the eager index enhanced INLJ with synchronous (INLJ-NS/S-Eager) and validation (INLJ) strategy require 767 and 172 seconds for index building, respectively, while other methods take less than 25 seconds. This substantial difference can be attributed to the increased frequency of validity checks for high-occurrence entries as skewness grows. Furthermore, Eager Index's practice of merging identical keys results in potentially very large entry sizes for high-occurrence items. This issue is exacerbated in the Synchronous approach, which must frequently retrieve these large entries to remove stale primary keys from the value. In contrast, Lazy Index and Composite Index, which don't eagerly merge entries, maintain more manageable entry sizes.

On the join latency side, Validation methods incur significantly higher costs, which exceed 4000 seconds when skewness approaches

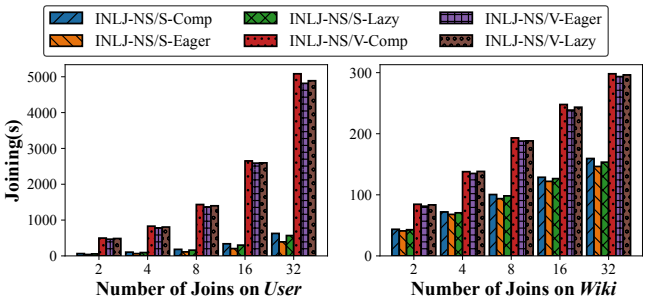


Figure 9: Different index designs deliver varied performance facing increasing join frequency.

0.7, while other methods take only a few hundred seconds. This can be explained by our analysis in Table 3, where each Validation method includes a lookup term of $O(d \cdot (L \cdot p + \lceil \frac{e}{B} \rceil))$. This term scales with increasing skewness and associated factor d, especially for INLJ, thus leading to the observed high join latencies. In summary, Lazy Index and Composite Index with Synchronous approach demonstrate superior resilience to data skewness.

4.7 Join frequency vs. Index design

We examine the impact of join frequency on index design by varying the join frequency on two datasets: *User* and *Wiki*, following the experiment in Section 4.3. As illustrated in Figure 9, the overall costs of Validation methods rise much faster than Synchronous methods as join frequency increases in both uniform (*Wiki*) and skewed (*User*) datasets. For example, in the *User* dataset, Validation costs jump from 460-500 seconds to 4800-5100 seconds, while Synchronous methods remain under 630 seconds. This difference stems from their design: Validation must check the data table for every matching entry, which results in repeated validity checks as join frequency increases. Synchronous methods, on the other hand, verify the validity of each entry only once during the index building phase, hence saving $O(d \cdot (L \cdot p + \lceil \frac{e}{B} \rceil))$ I/Os compared to Validation for each lookup in subsequent joins.

Moreover, Eager Index demonstrates the best overall performance among the three types of indexes. For instance, on the *User* dataset, Eager Index with Synchronous completes 32 joins in 390 seconds, while Composite and Lazy Index require 630 and 570 seconds. Eager Index saves 38% and 32% overall latency respectively, which will become more pronounced as join frequency increases. This advantage stems from Eager Index's lower join cost, which can be guaranteed as one entry lookup ($O(L' \cdot p + \lceil \frac{e'}{B} \rceil)$ as shown in Table 3). In contrast, the other two index types may scatter entries with the same keys across different levels of the LSM-tree, necessitating multiple retrievals for a single matching data entry. The performance gap seems relatively modest with Validation methods as their costs are dominated by the check process of Validation. In summary, Synchronous and Eager Index methods demonstrate superior performance as join operations become more frequent.

Remark: Eager Index and Validation are underestimated in conventional LSM-tree databases. Eager Index excels with frequent joins (e.g., more than twice per 10 million updates), while Validation outperforms Synchronous when entry sizes are large (e.g., over 512 bytes on the *Face* dataset).

4.8 Impact of increasing database updates

Database updates affect multiple parameters with complex system overhead impacts, making theoretical analysis difficult. Here we measured cumulative join and index building latencies across different join algorithms, index types, and consistency strategies with varying update counts. In our experiments, we tested both *Unif* and *Zipf* datasets by dividing the dataset into equal subsets, performing joins after processing each subset, and recording cumulative latencies, which results are shown in Figure 10.

In uniform datasets, cumulative join latency grows quadratically, meaning join costs increase linearly with data scale. This is supported by Table 4, which shows that almost every component is linearly related to the number of entries. For skewed datasets, as the duplicates of join attributes increases, the joining cost for INLJ methods exceeds that of SJ, and the cumulative latency trend increases more rapidly compared to uniform datasets. This is due to the exponential growth in duplicate join attributes, which necessitates significantly more point lookups. However, when the skewed attribute is the primary key, the impact is reduced since the duplicates are automatically merged by LSM-tree, thus reducing the data scale and the number of required point lookups.

In terms of index building, the cumulative latency increases linearly in both datasets, indicating that the cost of incrementally updating the index with each partition of uniform data remains consistent. As analysed in Table 3, the number of levels is almost the sole dynamic factor in each term, and these levels do not change rapidly during index construction. So the expected cost of index building remains relatively stable as the data volume increases. Furthermore, the index building cost for the Eager Index also climbs more steeply. This is because the frequently occurring join attributes produce enlarged index entries, which requires more resources to access.

Remark: In uniform datasets, join and index building costs grow steadily across methods, **thus our selection criteria is still applicable**. For highly skewed datasets, we recommend HJ or SJ with Composite or Lazy Index to deal with large-scale updates.

4.9 Impact of Tuning LSM-tree

To better understand the interaction of LSM-tree and joins, we examine various LSM-tree parameters. We utilize the flexible synthetic dataset with uniform distribution, *Unif*, to explicitly reflect the impact of different LSM-tree structures.

Size ratio. Figure 11 shows the impact of size ratio, a key compaction policy factor, which increases from 2 to 100. INLJ-NS/S-Comp join latency reduces from 60 to 30 seconds due to fewer LSM-tree levels and reduced Seek operations during lookups. However, Eager Index's join latency remains stable as it uses only one version of secondary keys. SJ and HJ are unaffected since lookup operations are not involved. Meanwhile, index building latency increases with size ratio. For SJ-SS/V-Comp, it rises from 25 to 105 seconds as size ratio increases from 20 to 100, due to heavier update overhead caused by larger compactions. For INLJ, larger size ratios benefit frequent join scenarios, while smaller ratios are better for infrequent joins. For SJ and HJ, smaller size ratios are recommended to reduce index building cost and improve join performance.

Write buffer. Memory allocation has garnered significant interest within the LSM-tree community [32, 48, 60]. We examined memory

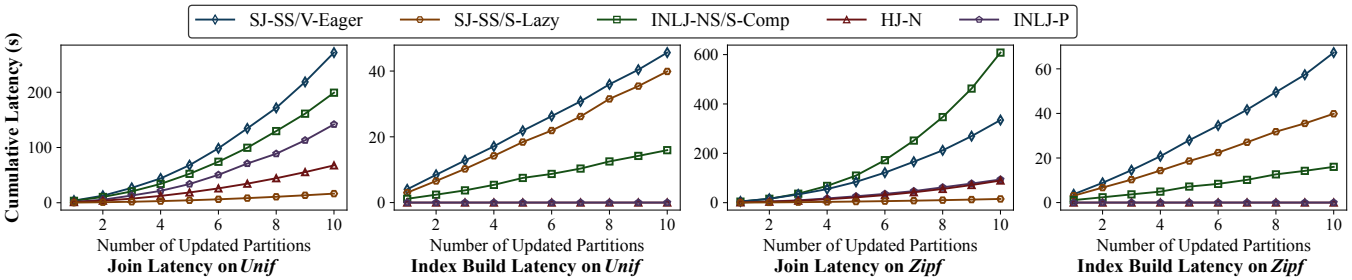


Figure 10: The performance of different join methods under increasing database updates.

allocation for the writer buffer, Bloom filters, and block cache, starting with the write buffer. In RocksDB, adjusting the write buffer size affects join latency for INLJ with Composite and Lazy Index by reducing levels and merging entries in index tables. This effect is less pronounced for Validation, where latency is predominantly influenced by the validation process. For SJ and HJ, as they do not rely on point lookups, changes in the number of levels do not affect their performance. While larger write buffers theoretically reduce index building latency, they can lead to inefficient flushes and I/O spikes. Figure 11 shows joining costs with 256MB buffer exceed those with 64MB buffer. We recommend using moderately large buffer sizes for better performance.

Bloom filter. Larger Bloom filter leads to reduced false positive rate that enhances INLJ method. As Figure 12 shows, the join latency of INLJ-NS/S-Eager reduces by 30% due to the drastically decreased false positive rate when we increase Bloom filter bits per key from 2 to 10. However, this improvement becomes marginal when we assign more memory, so we suggest using moderate memory (10 bits per key) to balance efficiency and memory usage.

Block cache. While larger block cache should reduce join latency for INLJ and Validation methods, Figure 12 shows an obvious start up phase due to the additional CPU overhead caused by frequent updates when small caches are involved. SJ without Validation and HJ are not affected by cache size since they perform full data scans. Hence, we recommend using either default page cache or large block cache for INLJ and Validation for better performance.

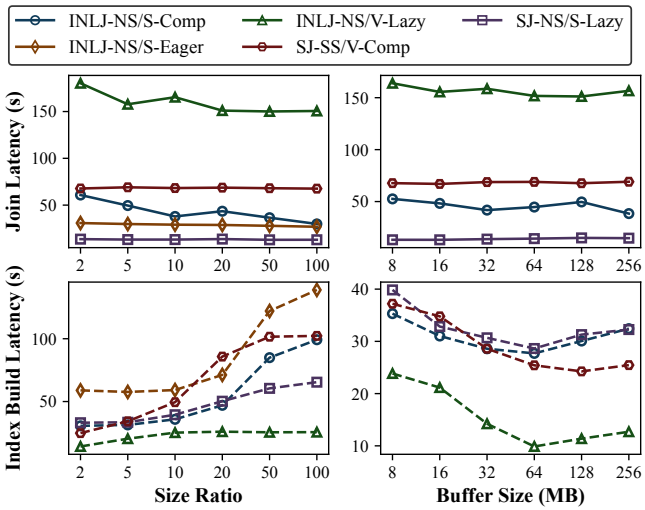


Figure 11: The impact of size ratio and write buffer.

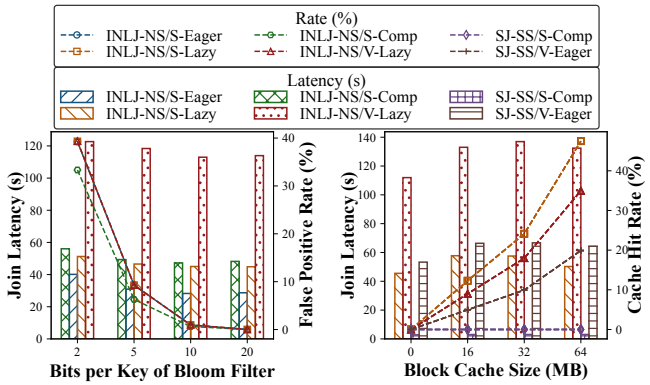


Figure 12: The impact of block cache size and Bloom filter.

Remark: Larger size ratios or writer buffers improve INLJ's joining performance but raise index building costs. So we suggest moderate size ratios (10) and writer buffers (64-128MB). For INLJ, 10 bits per key is sound for Bloom filters. Block cache can be skipped if sufficient page cache is available.

5 DISCUSSION

Here it is time to answer the question in title – *are joins over LSM-trees ready?* The experiments and analysis in this benchmark indicate that there is still considerable potential for optimizing join method selection. The join methods used by existing databases cover only a small portion of our proposed configuration space and often fail to achieve optimal performance. Additionally, the efficiency of different joins varies significantly under many conditions. For instance, in Figure 5 (b), SJ-PS could achieve more than 8x better latency than INLJ-P. Fortunately, we have derived valuable insights that enhance our understanding of this topic and offer practical takeaways for developers in selecting appropriate join methods. Moreover, we point out some constraints of our benchmark and suggest future directions to guide development.

5.1 Insights and takeaways

One Method Does Not Fit All. Figure 1 summarizes the selection strategies for different join components, demonstrating that each method could excel under specific conditions, with no single method dominating universally. For instance, HJ-P shows competitive performance in many scenarios as illustrated in Figure 5. However, it is outperformed by SJ-PS with frequent joins and by INLJ-P when dealing with large entry sizes. This also suggests an important takeaway message for joins over LSM-tree: ***Selection of join algorithms is influenced by multiple parameters (e.g.,***

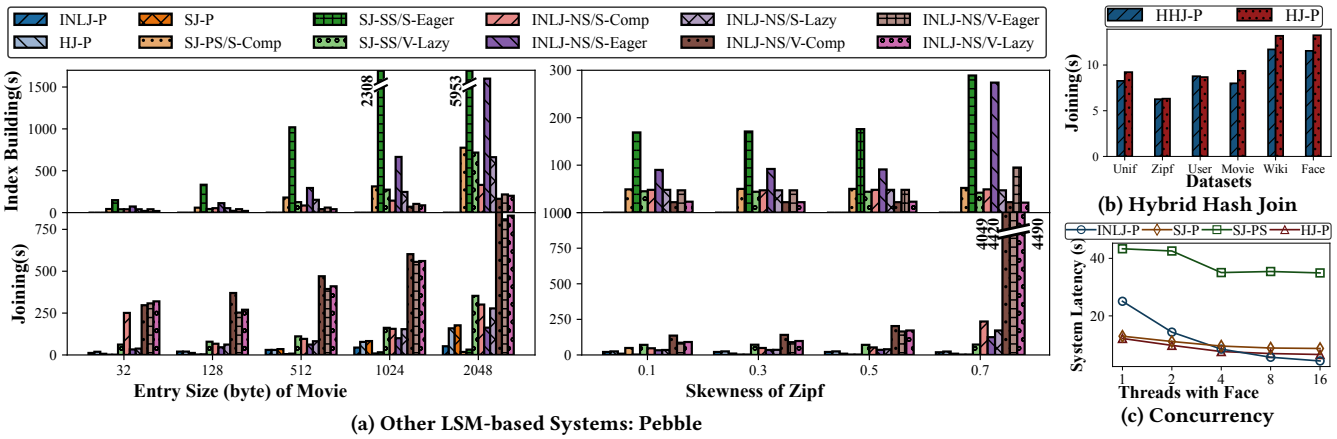


Figure 13: Case Studies of extending our insights to other system, hash join method, and concurrent execution.

entry size, join frequency, and data distribution), instead of only subject to selectivity.

Parameters matter unequally in join method selection. Among the parameters examined, data distribution, join frequency, and entry size are the most influential ones. ***Data distribution and join frequency are key for choosing secondary index types.*** Eager is preferred for frequent joins, excelling with skewed or highly duplicated join attributes, while Lazy and Comp offer lower index-building costs. ***Consistency strategies depend on entry size and join frequency.*** Synchronous with a covering index is ideal for frequent joins with small entries, whereas Validation with a non-covering index works better otherwise. For skewed data, avoid pairing Synchronous with Eager or Validation with INLJ due to excessive overhead. Other parameters, like Bloom filters, size ratio, and block cache size, affect performance but don't significantly alter join method priority. Identifying these key factors simplifies the selection of join methods.

Secondary indexes are not always necessary. Secondary indexes improve INLJ's join efficiency with Bloom filters and enhance SJ with reduced sorting costs. However, they come with certain index-building costs that sometimes can outweigh join improvements. As a result, secondary indexes may not always be beneficial, especially with large entries, high data skew, or infrequent joins.

5.2 Future Directions

Multi-way joins are significantly more complex than binary joins that we discussed. Nevertheless, multi-way joins can be decomposed into a sequence of binary joins [39, 53–56] thus our derived insights are still useful. For instance, we consider a join over three tables $R \bowtie S \bowtie W$, where $(R \bowtie S)$ aligns with our provided analysis and our insights still apply. This join produces an intermediate table (V) with N_V tuples, which should be stored in an LSM-tree before joining with W, leading to an overhead U_V of $O(L_V \cdot T \cdot N_V \cdot \frac{e_V}{B})$. It's straightforward to make V indexable to boost the subsequent join during LSM-tree construction. As a result, the costs of $(V \bowtie W)$ for INLJ, SJ, and HJ are $U_V + (N_W \cdot \frac{e_W}{B}) + N_W((1-\epsilon_W)Z_0(V) + \epsilon_W \cdot Z_1(V))$, $U_V + (N_V \cdot \frac{e_V}{B}) + 5(N_W \cdot \frac{e_W}{B})$, and $U_V + (3N_V \cdot \frac{e_V}{B}) + 3(N_W \cdot \frac{e_W}{B})$, respectively. This indicates that join method selection is heavily influenced by N_V. If N_V is much larger than N_W, the costs pertaining to W are negligible, indicating INLJ is preferable as it avoids extensive I/O overhead for reading the entire table V. If N_V is comparable to N_W, the selection remains consistent with our previous analysis. For small N_V, INLJ is still preferable and SJ could outperform HJ after introducing a secondary index. Meanwhile, INLJ remains the best due to the desirable lookup cost achieved by the Bloom filter. These reveal enhanced attractiveness of INLJ in multi-way joins over LSM-tree. Moreover, adding a secondary index for subsequent binary joins becomes less critical, especially when V is large, as an existing index is already sufficient in many scenarios. Additionally, many existing studies enhance join performance by optimizing binary join sequences to which our work could be adapted. Specifically, our theoretical models offer cost estimations and could be incorporated into optimization frameworks like Bao [40] to generate optimized query plans by applying the above analysis to different join orders.. The specialized schemes and index structures like multi-way hash trie join [40] are beyond the scope of this paper.

Moreover, LSM-based stores with different implementations and tuning parameters may present varied join performance thus yielding distinct insights, as our findings are based on experiments with RocksDB. However, we can easily extend our study to other platforms by adjusting configuration parameters and reevaluating the cost model for new insights. For example, platforms utilizing a key-value separation strategy like Wisckey [18, 46] offer higher update efficiency and may benefit from SJ due to faster index construction, which requires a reevaluation. In such cases, our proposed configuration space can still guide the design of new evaluation frameworks, and many of our insights remain applicable since they reveal the fundamental principles of join operations over LSM-trees.

Besides, we suggest several promising directions to improve joins over LSM-trees. One is optimizing join algorithms specifically for LSM-trees and modern hardware [72] to further boost performance. Another potential improvement lies in incorporating novel LSM-style [26] or hybrid secondary indexes [41] to reach flexible update and query trade-off and enhanced overall performance. Moreover, advanced consistency strategies to maintain data integrity while reducing the query or update overhead are also crucial. As data scales, distributed strategies and concurrent processing become vital for managing large datasets and high transaction volumes. Additionally, platform-specific optimizations, such as re-optimization schemes [41] and normalization approaches [17], could be further analyzed to identify the preferences for certain system.

5.3 Extending to other LSM-based Storages

We included another LSM-based system to validate the generality of the insights we provided. Among these systems, Pebble [17], the core engine of CockroachDB [71] written in Go, has garnered significant attention in the industry. Hence we use it to conduct a case study presented in Figure 13 (a). As expected, although the exact performance on Pebble differ from those on RocksDB, the relative differences in the costs of different join methods, as well as the trends in performance changes across different settings, are apparently consistent with the results observed on RocksDB. Specifically, the joining latency for the HJ and SJ methods increases almost proportionally with the entry size, whereas the INLJ methods exhibit a slower increase. For instance, when selectivity is moderate, INLJ-P surpasses HJ-P and SJ-P in joining efficiency when the entry size reaches 512B. These observations are consistent with the phenomena we observed in Figure 5 (a). Additionally, in Figure 13 (b), the joining latency for the INLJ methods and Validation increases significantly under higher skewness, due to more frequent retrieval of higher-frequency data. When skewness reaches 0.7, the joining latency for all INLJ methods with Validation exceeds 4000 seconds, whereas other methods remain below 200 seconds. At the same time, the index building times for both Eager Index and Synchronize are noticeably longer than those of other methods. These results are also similar to the findings from the RocksDB experiments in Figure 8. Therefore, despite differences in implementation, since our insights are based on the theoretical cost model of LSM-tree, they can be generalized to other LSM-based systems.

For the systems encompassing a subset of our configuration space, we can evaluate the associated space and utilize the corresponding insights or guidelines. In cases where join performance is altered by hybrid data structures or specific optimizations on certain join methods, we can adjust our findings to them with reevaluation since the instinct of each join method remains consistent.

5.4 Extending to other Hash Joins

To further evaluate the generality of the insights presented in this paper across more specialized join methods, we also included hybrid hash join [24, 68] as a case study. Compared to the default grace hash join used in this paper, hybrid hash join maintains an in-memory hash table for a portion of the table, potentially reducing I/O costs slightly. As shown in Figure 13 (b), across all datasets, hybrid hash join consistently exhibits a similar join cost to grace hash join, with a difference of only around one second. This is because joins over LSM-trees typically process massive data scale and performance is typically dependent on the I/O costs. In this case, the memory budget is significantly smaller than the total data volume, meaning hybrid hash join can only store a limited portion of data in memory, thus offering only marginal optimization. Therefore, the derived insights are still applicable for hybrid hash join, which further indicates the generality of our work.

5.5 Supplementary analysis on Concurrency

To further investigate the effects of potential variations in implementation, we examine the impact of concurrency in joins over LSM-trees on *Face*, a representative real dataset with uniform data distribution. Although concurrency does not appear to impact the I/O cost presented in Table 4, systems can still benefit from the parallel execution of join tasks which deserves further evaluation. To achieve this, we divide the key ranges approximately evenly across threads, allowing each thread to scan a different range concurrently. These scans allow HJ to build hash tables in parallel, SJ to sort data of each range before merging them, and INLJ to perform concurrent point queries on the target data table. As shown in Figure 13 (c), INLJ-P shows the best thread scaling, reducing latency by over 80% at 16 threads due to minimal thread interdependence - each thread independently scans RR table and performs point lookups on S table. SJ-P and HJ-P show less improvement as they are memory-bound, with threads waiting for memory resources. SJ-PS gains limited benefit as threading mainly affects index building compaction, which is heavily I/O-bound. Therefore, under memory constraints, INLJ methods demonstrate better concurrency potential.

6 RELATED WORK

LSM-tree stores. Research on LSM-trees focuses on optimizing parameters like size ratio, compaction policy, and Bloom filters through theoretical analysis [15, 19–22, 32, 33, 42, 45, 49, 59, 77]. Additionally, works focusing on self-designing data structures, like Cosine [14], Design Continuums [35], and Data Calculator [36], have been proposed to address specific challenges including varying data distribution, concurrency, and evolving hardware. We agree that these methods could potentially improve join performance, while they typically enhance system performance by including more data structures like B+ tree in their configuration space instead of focusing on optimizing a certain type. Meanwhile, our results can also be utilized in these works to analyze the LSM component and optimize toward join operations since our insights are general for LSM-tree structure. Moreover, various LSM-based industrial data stores [4, 5, 11, 13, 25, 28, 71] have supported join operation. While they typically provide several join methods for users to select by themselves. Thus, a comprehensive benchmark study of joins over LSM-trees is still lacking. The works focus on secondary indexes and consistency strategies [47, 65, 72] are also related.

Disk-based joins. Disk-based joins differ from in-memory joins [31, 66, 67] primarily due to their larger data volumes, necessitating more disk I/O operations. While extensively studied in relational databases [27, 38, 43, 44, 61, 64], traditional wisdom suggests that join costs mainly depend on selectivity or cardinality [3, 29, 70, 73, 76]. However, the LSM-tree context presents a different scenario. The unique structure of LSM-trees, compared to traditional storage engines like B+ trees [7, 35], results in distinct write and read I/O costs, significantly impacting join performance.

7 CONCLUSION

We provide a comprehensive analysis of LSM-tree join configurations and their theoretical foundations. Through extensive testing across various conditions and datasets, we derive several key LSM-specific insights and practical takeaways that could challenge current assumptions.

ACKNOWLEDGMENTS

This research is supported by NTU-NAP startup grant (022029-00001). We thank all reviewers for their valuable suggestions.

REFERENCES

[1] 2016. HBase. http://hbase.apache.org/. [Online; accessed 12-January-2022].

[2] 2021. WiredTiger. https://github.com/wiredtiger/wiredtiger.

[3] Ildar Absalyamov, Michael J Carey, and Vassilis J Tsotras. 2018. Lightweight cardinality estimation in LSM-based systems. In *Proceedings of the 2018 International Conference on Management of Data*. 841–855.

[4] Sattam Alsubaiee1 Yasser Altowim1 Hotham Altwaijry, Alexander Behm, Vinayak Borkar1 Yingyi Bu1 Michael Carey, Inci Cetindil1 Madhusudan Cheelangi, Khurram Faraaz, Eugenia Gabrielova1 Raman Grover1 Zachary Heilbron, Pouria Pirzadeh1 Vassilis Tsotras7 Rares Vernica, Jian Wen, and Till Westmann. 2014. AsterixDB: A Scalable, Open Source BDMS. *Proceedings of the VLDB Endowment* 7, 14 (2014).

[5] Apache. 2016. Cassandra. http://cassandra.apache.org.

[6] Timothy G Armstrong, Vamsi Ponnekanti, Dhruba Borthakur, and Mark Callaghan. 2013. Linkbench: a database benchmark based on the facebook social graph. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. 1185–1196.

[7] Rudolf Bayer and Edward McCreight. 1970. Organization and maintenance of large ordered indices. In *Proceedings of the 1970 ACM SIGFIDET (Now SIGMOD) Workshop on Data Description, Access and Control*. 107–141.

[8] Jon Bentley. 1984. Programming pearls: algorithm design techniques. *Commun. ACM* 27, 9 (1984), 865–873.

[9] Burton H Bloom. 1970. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM* 13, 7 (1970), 422–426.

[10] Edward Bortnikov, Anastasia Braginsky, Eshcar Hillel, Idit Keidar, and Gali Sheffi. 2018. Accordion: Better memory organization for LSM key-value stores. *Proceedings of the VLDB Endowment* 11, 12 (2018), 1863–1875.

[11] Wei Cao, Feifei Li, Gui Huang, Jianghang Lou, Jianwei Zhao, Dengcheng He, Mengshi Sun, Yingqiang Zhang, Sheng Wang, Xueqiang Wu, et al. 2022. PolarDB-X: An Elastic Distributed Relational Database for Cloud-Native Applications. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2859–2872.

[12] Zhao Cao, Shimin Chen, Feifei Li, Min Wang, and X Sean Wang. 2013. LogKV: Exploiting key-value stores for event log processing. In *Proc. Conf. Innovative Data Syst. Res.*

[13] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C Hsieh, Deborah A Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E Gruber. 2008. Bigtable: A distributed storage system for structured data. *ACM Transactions on Computer Systems (TOCS)* 26, 2 (2008), 1–26.

[14] Subarna Chatterjee, Meena Jagadeesan, Wilson Qin, and Stratos Idreos. 2021. Cosine: a cloud-cost optimized self-designing key-value storage engine. *Proceedings of the VLDB Endowment* 15, 1 (2021), 112–126.

[15] Guanduo Chen, Zhenying He, Meng Li, and Siqiang Luo. 2024. Oasis: An Optimal Disjoint Segmented Learned Range Filter. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1911–1924.

[16] Guoqiang Jerry Chen, Janet L Wiener, Shridhar Iyer, Anshul Jaiswal, Ran Lei, Nikhil Simha, Wei Wang, Kevin Wilfong, Tim Williamson, and Serhat Yilmaz. 2016. Realtime data processing at facebook. In *Proceedings of the 2016 International Conference on Management of Data*. 1087–1098.

[17] CockroachDB. 2020. Pebble. https://github.com/cockroachdb/pebble.

[18] Yifan Dai, Yien Xu, Aishwarya Ganesan, Ramnatthan Alagappan, Brian Kroth, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. 2020. From wisckey to bourbon: A learned index for log-structured merge trees. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*. 155–171.

[19] Niv Dayan, Manos Athanassoulis, and Stratos Idreos. 2017. Monkey: Optimal navigable key-value store. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 79–94.

[20] Niv Dayan and Stratos Idreos. 2018. Dostoevsky: Better space-time trade-offs for LSM-tree based key-value stores via adaptive removal of superfluous merging. In *Proceedings of the 2018 International Conference on Management of Data*. 505–520.

[21] Niv Dayan, Tamar Weiss, Shmuel Dashevsky, Michael Pan, Edward Bortnikov, and Moshe Twitto. 2022. Spooky: granulating LSM-tree compactions correctly. *Proceedings of the VLDB Endowment* 15, 11 (2022), 3071–3084.

[22] Dayan, Niv and Idreos, Stratos. 2019. The log-structured merge-bush & the wacky continuum. In *Proceedings of the 2019 International Conference on Management of Data*. 449–466.

[23] Giuseppe DeCandia, D Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, S Sivasubramanian A Pilchin, P Vosshall, and W Vogels. 2007. Dynamo: Amazon's Highly Available Key-Value Store. SOSP, 2007.

[24] David J DeWitt, Randy H Katz, Frank Olken, Leonard D Shapiro, Michael R Stonebraker, and David A Wood. 1984. Implementation techniques for main memory database systems. In *Proceedings of the 1984 ACM SIGMOD international conference on management of data*. 1–8.

[25] Facebook. 2016. RocksDB. https://github.com/facebook/rocksdb.

[26] Paolo Ferragina and Giorgio Vinciguerra. 2020. The PGM-index: a fully-dynamic compressed learned index with provable worst-case bounds. *Proceedings of the VLDB Endowment* 13, 8 (2020), 1162–1175.

[27] Kevin P Gaffney, Martin Prammer, Larry Brasfield, D Richard Hipp, Dan Kennedy, and Jignesh M Patel. 2022. Sqlite: past, present, and future. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3535–3547.

[28] Google. 2016. LevelDB. https://github.com/google/leveldb/.

[29] Benjamin Hilprecht, Andreas Schmidt, Moritz Kulessa, Alejandro Molina, Kristian Kersting, and Carsten Binnig. 2019. DeepDB: Learn from Data, not from Queries! *Proceedings of the VLDB Endowment* 13, 7 (2019).

[30] Gui Huang, Xuntao Cheng, Jianying Wang, Yujie Wang, Dengcheng He, Tieying Zhang, Feifei Li, Sheng Wang, Wei Cao, and Qiang Li. 2019. X-Engine: An optimized storage engine for large-scale E-commerce transaction processing. In *Proceedings of the 2019 International Conference on Management of Data*. 651–665.

[31] Wentao Huang, Yunhong Ji, Xuan Zhou, Bingsheng He, and Kian-Lee Tan. 2023. A Design Space Exploration and Evaluation for Main-Memory Hash Joins in Storage Class Memory. *Proceedings of the VLDB Endowment* 16, 6 (2023), 1249–1263.

[32] Andy Huynh, Harshal A Chaudhari, Evimaria Terzi, and Manos Athanassoulis. 2021. Endure: A Robust Tuning Paradigm for LSM Trees Under Workload Uncertainty. *arXiv preprint arXiv:2110.13801* (2021).

[33] Andy Huynh, Harshal A. Chaudhari, Evimaria Terzi, and Manos Athanassoulis. 2023. Towards Flexibility and Robustness of LSM Trees. arXiv:2311.10005 [cs.DB]

[34] Andy Huynh, Harshal A Chaudhari, Evimaria Terzi, and Manos Athanassoulis. 2024. Towards flexibility and robustness of LSM trees. *The VLDB Journal* (2024), 1–24.

[35] Stratos Idreos, Niv Dayan, Wilson Qin, Mali Akmanalp, Sophie Hilgard, Andrew Ross, James Lennon, Varun Jain, Harshita Gupta, David Li, et al. 2019. Design Continuums and the Path Toward Self-Designing Key-Value Stores that Know and Learn.. In *CIDR*.

[36] Stratos Idreos, Kostas Zoumpatianos, Brian Hentschel, Michael S Kester, and Demi Guo. 2018. The data calculator: Data structure design and cost synthesis from first principles and learned cost models. In *Proceedings of the 2018 International Conference on Management of Data*. 535–550.

[37] William Jannen, Jun Yuan, Yang Zhan, Amogh Akshintala, John Esmet, Yizheng Jiao, Ankur Mittal, Prashant Pandey, Phaneendra Reddy, Leif Walsh, et al. 2015. BetrFS: A Right-Optimized Write-Optimized File System.. In *FAST*, Vol. 15. 301–315.

[38] Christopher Jermaine, Alin Dobra, Subramanian Arumugam, Shantanu Joshi, and Abhijit Pol. 2005. A disk-based join with probabilistic guarantees. In *Proceedings of the 2005 ACM SIGMOD international conference on Management of data*. 563–574.

[39] David Justen, Daniel Ritter, Campbell Fraser, Andrew Lamb, Allison Lee, Thomas Bodner, Mhd Yamen Haddad, Steffen Zeuch, Volker Markl, and Matthias Boehm. 2024. POLAR: Adaptive and Non-invasive Join Order Selection via Plans of Least Resistance. *Proceedings of the VLDB Endowment* 17, 6 (2024), 1350–1363.

[40] Ahmad Khazaie and Holger Pirk. 2023. SonicJoin: Fast, Robust and Worst-case Optimal.. In *EDBT*. 540–551.

[41] Taewoo Kim, Alexander Behm, Michael Blow, Vinayak Borkar, Yingyi Bu, Michael J Carey, Murtadha Hubail, Shiva Jahangiri, Jianfeng Jia, Chen Li, et al. 2020. Robust and efficient memory management in Apache AsterixDB. *Software: Practice and Experience* 50, 7 (2020), 1114–1151.

[42] Eric R Knorr, Baptiste Lemaire, Andrew Lim, Siqiang Luo, Huanchen Zhang, Stratos Idreos, and Michael Mitzenmacher. 2022. Proteus: A Self-Designing Range Filter. In *Proceedings of the 2022 International Conference on Management of Data*. 1670–1684.

[43] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How good are query optimizers, really? *Proceedings of the VLDB Endowment* 9, 3 (2015), 204–215.

[44] Viktor Leis, Bernhard Radke, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2018. Query optimization through the looking glass, and what we found running the join order benchmark. *The VLDB Journal* 27, 5 (2018), 643–668.

[45] Junfeng Liu, Fan Wang, Dingheng Mo, and Siqiang Luo. 2024. Structural Designs Meet Optimality: Exploring Optimized LSM-tree Structures in A Colossal Configuration Space. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–26.

[46] Lanyue Lu, Thanumalayan Sankaranarayana Pillai, Hariharan Gopalakrishnan, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. 2017. Wisckey: Separating keys from values in ssd-conscious storage. *ACM Transactions on Storage (TOS)* 13, 1 (2017), 1–28.

[47] Chen Luo and Michael J Carey. 2019. Efficient data ingestion and query processing for LSM-based storage systems. *Proceedings of the VLDB Endowment* 12, 5 (2019), 531–543.

[48] Chen Luo and Michael J Carey. 2020. Breaking down memory walls: Adaptive memory management in LSM-based storage systems. *Proceedings of the VLDB Endowment* 14, 3 (2020), 241–254.

[49] Siqiang Luo, Subarna Chatterjee, Rafael Ketsetsidis, Niv Dayan, Wilson Qin, and Stratos Idreos. 2020. Rosetta: A robust space-time optimized range filter for key-value stores. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 2071–2086.

[50] Ryan Marcus, Andreas Kipf, Alexander van Renen, Mihail Stoia, Sanchit Misra, Alfons Kemper, Thomas Neumann, and Tim Kraska. 2021. Flow-loss: Learning cardinality estimates that matter. *Proceedings of the VLDB Endowment* 14, 11 (2021).

[51] Ryan Marcus, Andreas Kipf, Alexander van Renen, Mihail Stoian, Sanchit Misra, Alfons Kemper, Thomas Neumann, and Tim Kraska. 2020. Benchmarking learned indexes. *Proceedings of the VLDB Endowment* 14, 1 (2020), 1–13.

[52] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2020. Bao: Learning to steer query optimizers. *arXiv preprint arXiv:2004.03814* (2020).

[53] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making learned query optimization practical. In *Proceedings of the 2021 International Conference on Management of Data*. 1275–1288.

[54] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. 2019. Neo: a learned query optimizer. *Proceedings of the VLDB Endowment* 12, 11 (2019), 1705–1718.

[55] Ryan Marcus and Olga Papaemmanouil. [n.d.]. Plan-Structured Deep Neural Network Models for Query Performance Prediction. *Proceedings of the VLDB Endowment* 12, 11 ([n. d.]).

[56] Ryan Marcus and Olga Papaemmanouil. 2018. Deep reinforcement learning for join order enumeration. In *Proceedings of the First International Workshop on Exploiting Artificial Intelligence Techniques for Data Management*. 1–4.

[57] Yoshinori Matsunobu, Siying Dong, and Herman Lee. 2020. Myrocks: Lsm-tree database storage engine serving facebook's social graph. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3217–3230.

[58] Scott Meyers. 2001. *Effective STL: 50 specific ways to improve your use of the standard template library*. Pearson Education.

[59] Dingheng Mo, Fanchao Chen, Siqiang Luo, and Caihua Shan. 2023. Learning to Optimize LSM-trees: Towards A Reinforcement Learning based Key-Value Store for Dynamic Workloads. *Proceedings of the ACM on Management of Data* 1, 3 (2023), 1–25.

[60] Ju Hyoung Mun, Zichen Zhu, Aneesh Raman, and Manos Athanassoulis. 2022. LSM-Trees Under (Memory) Pressure. In *Proceedings of the International Workshop on Accelerating Data Management Systems Using Modern Processor and Storage Architectures (ADMS)*.

[61] MySQL. 2022. MySQL. *Web resource: https://www.mysql.com/* (2022).

[62] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. 1996. The log-structured merge-tree (LSM-tree). *Acta Informatica* 33 (1996), 351–385.

[63] Arash Partow. 2013. General purpose hash function algorithms. (2013).

[64] Behandelt PostgreSQL. 2022. PostgreSQL. *Web resource: http://www.PostgreSQL.org/* (2022).

[65] Mohiuddin Abdul Qader, Shiwen Cheng, and Vagelis Hristidis. 2018. A comparative study of secondary indexing techniques in LSM-based NoSQL databases. In *Proceedings of the 2018 International Conference on Management of Data*. 551–566.

[66] Ibrahim Sabek and Tim Kraska. 2023. The Case for Learned In-Memory Joins. *Proceedings of the VLDB Endowment* 16, 7 (2023), 1749–1762.

[67] Stefan Schuh, Xiao Chen, and Jens Dittrich. 2016. An experimental comparison of thirteen relational equi-joins in main memory. In *Proceedings of the 2016 International Conference on Management of Data*. 1961–1976.

[68] Leonard D Shapiro. 1986. Join processing in database systems with large main memories. *ACM Transactions on Database Systems (TODS)* 11, 3 (1986), 239–264.

[69] Pradeep J Shetty, Richard P Spillane, Ravikant R Malpani, Binesh Andrews, Justin Seyster, and Erez Zadok. 2013. Building workload-independent storage with VT-trees. In *Presented as part of the 11th {USENIX} Conference on File and Storage Technologies ({FAST} 13)*. 17–30.

[70] Ji Sun and Guoliang Li. 2018. An End-to-End Learning-based Cost Estimator. *Proceedings of the VLDB Endowment* 13, 3 (2018).

[71] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, et al. 2020. Cockroachdb: The resilient geo-distributed sql database. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1493–1509.

[72] Jing Wang, Youyou Lu, Qing Wang, Yuhao Zhang, and Jiwu Shu. 2023. Revisiting Secondary Indexing in {LSM-based} Storage Systems with Persistent Memory. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 817–832.

[73] Peizhi Wu and Gao Cong. 2021. A unified deep model of learning from both data and queries for cardinality estimation. In *Proceedings of the 2021 International Conference on Management of Data*. 2009–2022.

[74] Wentao Wu, Yun Chi, Shenghuo Zhu, Junichi Tatemura, Hakan Hacigümüs, and Jeffrey F Naughton. 2013. Predicting query execution time: Are optimizer cost models really unusable?. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. IEEE, 1081–1092.

[75] Ziniu Wu, Parimarjan Negi, Mohammad Alizadeh, Tim Kraska, and Samuel Madden. 2023. FactorJoin: a new cardinality estimation framework for join queries. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27.

[76] Zongheng Yang, Amog Kamsetty, Sifei Luan, Eric Liang, Yan Duan, Xi Chen, and Ion Stoica. 2020. NeuroCard: one cardinality estimator for all tables. *Proceedings of the VLDB Endowment* 14, 1 (2020), 61–73.

[77] Weiping Yu, Siqiang Luo, Zihao Yu, and Gao Cong. 2024. CAMAL: Optimizing LSM-trees via Active Learning. *Proceedings of the ACM on Management of Data* 2, 4 (2024), 1–26.